Revised Prioritized Project and Study Plan

Stephen MacKenzie

Prioritized Plan of Action

1. Highest Priority: MLibs Project Video

- **Objective**: Create and upload a video demonstrating the MLibs project, including its setup, components, and real-time execution.
- **Tasks**:
  - Prepare a video script outlining the introduction, project overview, and real-time execution of the MLibs project.
  - Record the setup, build, and execution process using screen recording software.
  - Include an explanation of each library module (priority queues, hash tables, etc.) with Doxygen documentation as visual aids.
  - Demonstrate the automated build, flash, and run process using the `bfr.py` script.
  - Upload the completed video to YouTube and share it on LinkedIn.
- **Timeframe**: 1-2 days

2. Real-Time Packet Reading and Interaction (Voltage Analyzer Project)

- **Objective**: Enhance the `voltage_analyzer` project to read Ethernet packets in real-time from the microcontroller and interact with the microcontroller using two-way communication.
- **Tasks**:
  - Implement a thread to handle real-time packet reading using non-blocking socket I/O and 'ncurses' for display.
  - Set up a thread for sending packets to the microcontroller, triggering interrupts for polling data.
  - Implement interrupt handling and threading to synchronize data processing.
  - Integrate temperature monitoring code, making the application a comprehensive remote monitor and analyzer.
  - Add Doxygen support to generate documentation for each module and provide detailed comments for better code understanding.
- **Outcome**: A real-time remote monitor and analyzer that reads and displays voltage and temperature data from the microcontroller, showcasing multi-threaded interactions and low-level network programming.
- **Timeframe**: 1 week

3. Multithreaded Development Assignment

- **Objective**: Implement a multi-threaded data pipeline that utilizes various threading primitives for synchronization and coordination.
- **Tasks**:

  - Develop a data processing pipeline with multiple threads, each handling different stages of data transformation (e.g., reading, processing, and aggregating).
  - Implement synchronization using `std::mutex`, `std::atomic`, and `std::condition_variable`.
  - Add demonstrations of thread-safe data structures and synchronization methods.
  - Integrate `std::numerics` components for computational tasks like FFT.
- **Outcome**: A comprehensive multi-threaded project that showcases your understanding of synchronization and parallelism in C++.
- **Timeframe**: 2-3 weeks

4. Numerics Library Integration

- **Objective**: Understand and implement numerics operations using `std::numeric` and related libraries to complement threading work.
- **Tasks**:
  - Explore and implement key numerics operations (e.g., `std::valarray`, `std::complex`, `std::accumulate`).
  - Integrate numerics computations into the threading project (e.g., using FFT for data processing).
  - Profile and optimize numerics operations in multi-threaded scenarios.
- **Outcome**: A strong grasp of numerics utilities and how they can be used effectively in concurrent environments.
- **Timeframe**: 1-2 weeks

5. CLRS-Based Assignments (Lower Priority)

- **Objective**: Reinforce algorithmic problem-solving skills with targeted exercises from CLRS.
- **Tasks**:
  - Implement dynamic programming exercises like the Rod Cutting problem.
  - Work on graph theory problems such as BFS, DFS, and concurrent graph traversal algorithms.
  - Implement concurrent versions of algorithms (e.g., parallel Dijkstra's shortest path) using threading concepts.
- **Outcome**: A solid understanding of algorithms that complement your threading and numerics work.
- **Timeframe**: As time permits, revisit after completing the primary threading and numerics work.

6. Additional Stretch Goals (Optional)

- **Objective**: Implement advanced projects that leverage custom allocators, memory management, and real-world simulations.
- **Tasks**:
  - Implement a memory pool allocator for multi-threaded allocation and deallocation.
  - Simulate real-world scenarios like traffic management using threading and graph algorithms.
- **Outcome**: A deeper understanding of memory management and concurrency in real-world systems.
- **Timeframe**: Optional, if time permits.